

MAST32001 Computational Statistics - Lecture 4

Monte Carlo methods and multivariate simulation

Klaus Nordhausen

University of Helsinki

WS 2026

Today

- ▶ Monte Carlo as a computational principle
- ▶ Monte Carlo integration
- ▶ Monte Carlo error and convergence
- ▶ Simulation studies in statistics
- ▶ Importance sampling
- ▶ Spherical distributions
- ▶ Uniform simulation on the sphere
- ▶ Elliptical distributions and multivariate normal simulation

Monte Carlo methods

What is a Monte Carlo method?

Monte Carlo methods use repeated random sampling to obtain numerical results.

Applications include

- ▶ numerical integration;
- ▶ probabilities that are difficult to calculate analytically;
- ▶ bias, variance and MSE of estimators;
- ▶ confidence interval coverage;
- ▶ power of statistical tests;
- ▶ optimization;
- ▶ sampling from complex distributions.

The random numbers are a **computational tool**.

Historical motivation

The name is associated with work by Stanislaw Ulam, John von Neumann and others in the 1940s.

A key idea is simple:

If an analytical calculation is difficult, perhaps the quantity can be represented as an expectation and approximated by repeated simulation.

This reverses the usual viewpoint: probability can be used to solve deterministic numerical problems.

Backbone of a Monte Carlo simulation

1. Define the quantity of interest.
2. Specify a probability distribution for simulated inputs.
3. Generate random inputs.
4. Perform the deterministic calculation.
5. Aggregate the outputs.
6. Quantify the remaining Monte Carlo uncertainty.

Step 6 is essential: a Monte Carlo answer is itself an estimate.

Monte Carlo integration

Expectations are integrals

Let X have density f . Then

$$E_f[h(X)] = \int h(x)f(x) dx.$$

If

$$X_1, \dots, X_m \stackrel{iid}{\sim} f,$$

we estimate this expectation by

$$\hat{\theta}_m = \frac{1}{m} \sum_{i=1}^m h(X_i).$$

Why Monte Carlo integration works

Let

$$Y_i = h(X_i).$$

If $E|Y_i| < \infty$, the strong law of large numbers gives

$$\hat{\theta}_m = \bar{Y}_m \xrightarrow{a.s.} E(Y_i) = \theta.$$

Thus increasing the Monte Carlo sample size makes the approximation consistent.

Monte Carlo variance

If

$$\sigma_h^2 = \text{Var}\{h(X)\} < \infty,$$

then

$$\text{Var}(\hat{\theta}_m) = \frac{\sigma_h^2}{m}.$$

Hence

$$\text{sd}(\hat{\theta}_m) = \frac{\sigma_h}{\sqrt{m}}.$$

The characteristic Monte Carlo convergence rate is therefore

$$\boxed{m^{-1/2}}.$$

Monte Carlo standard error

Estimate σ_h^2 by

$$s_h^2 = \frac{1}{m-1} \sum_{i=1}^m \{h(X_i) - \hat{\theta}_m\}^2.$$

Then

$$\widehat{\text{se}}_{\text{MC}}(\hat{\theta}_m) = \frac{s_h}{\sqrt{m}}.$$

This measures uncertainty due to using a **finite Monte Carlo sample**.

Central limit theorem

Under standard conditions,

$$\sqrt{m} \frac{\hat{\theta}_m - \theta}{\sigma_h} \Rightarrow N(0, 1).$$

Thus an approximate Monte Carlo confidence interval is

$$\hat{\theta}_m \pm 1.96 \hat{\text{se}}_{\text{MC}}.$$

This interval concerns the numerical Monte Carlo approximation, not necessarily the original statistical parameter.

Statistical error versus Monte Carlo error

Suppose data D give an estimator $\hat{\theta}(D)$, but evaluating it requires simulation.

There can be two distinct sources of uncertainty:

statistical sampling error

from observing a finite data set, and

Monte Carlo error

from using a finite number of simulation draws.

Increasing m reduces Monte Carlo error. It does **not** create more information in the observed data.

Example: $\int_0^1 x^4 dx$

Since $U \sim U(0, 1)$,

$$E(U^4) = \int_0^1 x^4 dx = \frac{1}{5}.$$

```
set.seed(20260812)
m <- 10000
u <- runif(m)
h <- u^4
estimate <- mean(h)
mc_se <- sd(h) / sqrt(m)
c(estimate = estimate,
  mc_se = mc_se,
  exact = 1/5)
```

```
##      estimate      mc_se      exact
## 0.197777797 0.002648033 0.200000000
```

How much simulation is enough?

Since

$$\text{se}_{MC} \propto \frac{1}{\sqrt{m}},$$

reducing the Monte Carlo standard error by a factor of

- ▶ 2 requires $4m$ simulations;
- ▶ 10 requires $100m$ simulations;
- ▶ 100 requires $10\,000m$ simulations.

Brute-force precision can become computationally expensive.

R TASK: watch Monte Carlo error shrink

Estimate

$$E(U^4) = 1/5$$

using

$$m = 10^2, 10^3, 10^4, 10^5.$$

For each m , compute

- ▶ the Monte Carlo estimate;
- ▶ its absolute error;
- ▶ its estimated Monte Carlo standard error.

Does multiplying m by 100 reduce the typical error by about 10?

Solution: Monte Carlo error

```
set.seed(20260812)
ms <- c(1e2, 1e3, 1e4, 1e5)

res <- do.call(rbind, lapply(ms, function(m) {
  u <- runif(m)
  h <- u^4
  est <- mean(h)
  c(m = m,
    estimate = est,
    abs_error = abs(est - 1/5),
    mc_se = sd(h) / sqrt(m))
}))
```

Solution: Monte Carlo error

```
res
```

```
##           m estimate   abs_error   mc_se
## [1,] 1e+02 0.2084306 0.0084306098 0.0265907230
## [2,] 1e+03 0.2100376 0.0100376102 0.0087309565
## [3,] 1e+04 0.1988531 0.0011469353 0.0026476237
## [4,] 1e+05 0.1996146 0.0003854071 0.0008442119
```

A single realization need not decrease monotonically, but the **typical scale** of the error is $m^{-1/2}$.

Integrals over arbitrary finite intervals

To compute

$$I = \int_a^b h(x) dx,$$

let $X \sim U(a, b)$. Its density is $1/(b - a)$, so

$$E[h(X)] = \frac{1}{b - a} \int_a^b h(x) dx.$$

Therefore

$$I = (b - a)E[h(X)].$$

Example: $\int_2^5 \sin(x) dx$

Let

$$I = \int_2^5 \sin(x) dx.$$

If $X \sim U(2, 5)$, then

$$E[\sin(X)] = \frac{1}{5-2} \int_2^5 \sin(x) dx,$$

and therefore

$$I = (5-2)E[\sin(X)].$$

We can now compare **Monte Carlo integration**, R's **numerical integration**, and the **exact analytical result**.

Example: $\int_2^5 \sin(x) dx$

```
set.seed(20260812)

m <- 100000
x <- runif(m, min = 2, max = 5)
# Monte Carlo integration
mc <- (5 - 2) * mean(sin(x))
# Numerical integration
num <- integrate(sin, lower = 2, upper = 5)$value
# Analytical result
exact <- -cos(5) + cos(2)

c(Monte_Carlo = mc,
  Numerical = num,
  Exact = exact)
```

```
## Monte_Carlo    Numerical      Exact
## -0.6950927    -0.6998090    -0.6998090
```

Higher-dimensional integration

Let U and V be independent $U(0, 1)$. Then

$$E[h(U, V)] = \int_0^1 \int_0^1 h(u, v) du dv.$$

The same principle extends to p dimensions:

$$E[h(U_1, \dots, U_p)] = \int_{[0,1]^p} h(u) du.$$

This is one reason Monte Carlo becomes attractive in high-dimensional integration.

Estimating π

Generate

$$X, Y \stackrel{iid}{\sim} U(0, 1)$$

and define

$$I = 1\{X^2 + Y^2 \leq 1\}.$$

Then

$$P(I = 1) = \frac{\pi}{4}$$

and

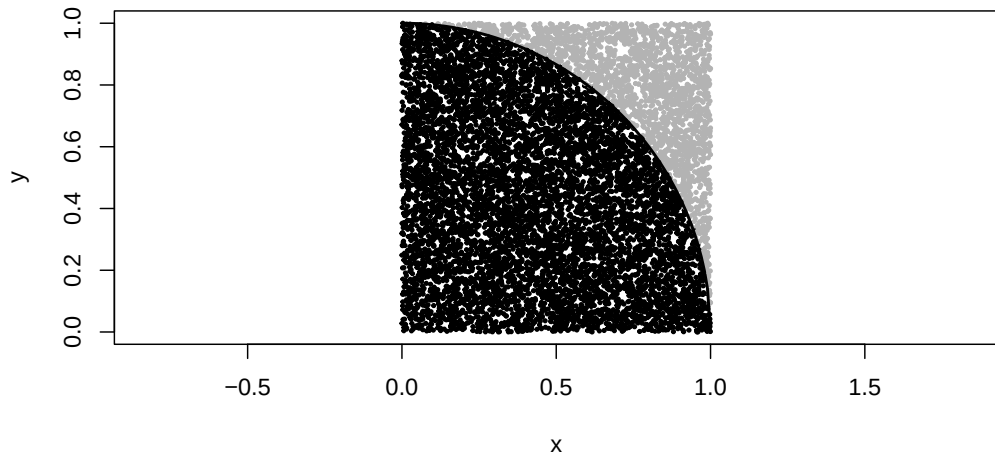
$$\hat{\pi} = 4\bar{I}.$$

Estimating π

```
set.seed(20260812)
m <- 10000
x <- runif(m)
y <- runif(m)
inside <- x^2 + y^2 <= 1
c(pi_hat = 4 * mean(inside),
  mc_se = 4 * sd(inside) / sqrt(m))
```

```
##      pi_hat      mc_se
## 3.17000000 0.01622148
```

Visualizing the π experiment



Monte Carlo in statistical studies

Estimating MSE

For an estimator $\hat{\theta}$,

$$\text{MSE}(\hat{\theta}) = E\{(\hat{\theta} - \theta)^2\}.$$

Generate m independent data sets and compute estimates

$$\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(m)}.$$

Then

$$\widehat{\text{MSE}} = \frac{1}{m} \sum_{j=1}^m (\hat{\theta}^{(j)} - \theta)^2.$$

MSE of the sample median

```
set.seed(20260812)

n <- 30
m <- 5000
theta <- 0
theta_hat <- replicate(
  m,
  median(rnorm(n, mean = theta, sd = 1))
)

c(bias = mean(theta_hat) - theta,
  variance = var(theta_hat),
  mse = mean((theta_hat - theta)^2))
```

```
##          bias      variance          mse
## -0.001000448  0.048930038  0.048921253
```

Bias-variance decomposition

Recall

$$\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

A simulation study can estimate all three quantities.

But each estimated performance measure also has Monte Carlo uncertainty.

A simulation study is itself a statistical experiment.

Confidence interval coverage

Suppose a procedure produces a nominal 95% interval

$$[L(D), U(D)].$$

Coverage is

$$P_{\theta}\{L(D) \leq \theta \leq U(D)\}.$$

In a simulation study, define

$$I_j = 1\{L(D_j) \leq \theta \leq U(D_j)\}.$$

Then estimated coverage is simply

$$\hat{C} = \frac{1}{m} \sum_{j=1}^m I_j.$$

Monte Carlo uncertainty of coverage

Since I_j is Bernoulli,

$$\text{se}_{MC}(\hat{C}) \approx \sqrt{\frac{\hat{C}(1 - \hat{C})}{m}}.$$

If true coverage is around 0.95 and $m = 1000$,

$$\text{se}_{MC} \approx \sqrt{\frac{0.95(0.05)}{1000}} \approx 0.0069.$$

So an observed coverage of 0.94 versus 0.95 may simply be Monte Carlo noise.

Power by simulation

For a test with rejection indicator

$$I_j = 1\{\text{reject } H_0\},$$

simulate data under a specified alternative.

Then

$$\widehat{\text{power}} = \frac{1}{m} \sum_{j=1}^m I_j.$$

Again this is a binomial proportion, so its Monte Carlo standard error is easy to estimate.

Importance sampling

General Monte Carlo representation

Suppose the target is

$$I = \int h(x) dx.$$

For any density g with suitable support,

$$I = \int \frac{h(x)}{g(x)} g(x) dx = E_g \left[\frac{h(X)}{g(X)} \right].$$

Therefore, for $X_i \sim g$,

$$\hat{I} = \frac{1}{m} \sum_{i=1}^m \frac{h(X_i)}{g(X_i)}.$$

Why the choice of g matters

The variance is

$$\mathrm{Var}_g \left\{ \frac{h(X)}{g(X)} \right\}.$$

A poor g can make the weights

$$w(x) = \frac{h(x)}{g(x)}$$

extremely variable.

A useful importance distribution places substantial probability where the integrand contributes strongly.

This is a first example of **variance reduction**.

Example with an infinite interval

Consider

$$I = \int_1^{\infty} e^{-x^2} dx.$$

Write $x = y + 1$:

$$I = \int_0^{\infty} e^{-(y+1)^2} dy.$$

If $Y \sim \text{Exp}(1)$,

$$I = E \left[\frac{e^{-(Y+1)^2}}{e^{-Y}} \right].$$

```
set.seed(20260812)
y <- rexp(100000)
mc <- mean(exp(-(y + 1)^2) / dexp(y))
exact <- integrate(function(x) exp(-x^2), 1, Inf)$value
c(Monte_Carlo = mc, numerical = exact)
```

```
## Monte_Carlo    numerical
##    0.1392508    0.1394028
```

Multivariate random generation

Why multivariate simulation is different

For a univariate variable, we mainly need to reproduce one distribution.

For a vector

$$X = (X_1, \dots, X_p)^\top,$$

we must reproduce both

- ▶ marginal behaviour;
- ▶ dependence between coordinates.

Generating each coordinate from the correct marginal distribution is not enough.

Spherical symmetry

A random vector $X \in \mathbb{R}^p$ is **spherically symmetric** if

$$QX \stackrel{d}{=} X$$

for every orthogonal matrix Q , where

$$Q^\top Q = I.$$

The distribution is unchanged by rotations and reflections.

Radius and direction

For $X \neq 0$, write

$$R = \|X\|_2, \quad U = \frac{X}{\|X\|_2}.$$

Then

$$X = RU.$$

Here

$$R \geq 0$$

describes distance from the origin, while

$$U \in \mathbb{S}^{p-1}$$

describes direction.

For a spherical distribution, the direction is uniform on the sphere and independent of the radius.

The unit sphere

$$\mathbb{S}^{p-1} = \{u \in \mathbb{R}^p : u^\top u = 1\}.$$

In dimensions:

- ▶ $p = 2$: $\mathbb{S}^1$, the unit circle;
- ▶ $p = 3$: $\mathbb{S}^2$, the ordinary sphere;
- ▶ general p : the unit sphere in $\mathbb{R}^p$.

We need an algorithm for sampling U uniformly from this set.

A tempting but wrong idea

Generate each coordinate independently from $U(-1, 1)$ and normalize:

$$V_j \stackrel{iid}{\sim} U(-1, 1), \quad U = \frac{V}{\|V\|}.$$

This is **not** uniform on the sphere.

The original cube is not rotationally invariant, so normalization preserves directional preferences.

Uniform marginals do not imply uniform directions.

Uniform on the sphere from normals

Generate

$$Z \sim N_p(0, I_p)$$

and define

$$U = \frac{Z}{\|Z\|_2}.$$

Then

$$U \sim \text{Unif}(\mathbb{S}^{p-1}).$$

Why?

The standard multivariate normal is rotationally invariant:

$$QZ \stackrel{d}{=} Z$$

for every orthogonal Q .

Therefore its direction cannot prefer any orientation.

Radius and direction of a standard normal

Let

$$Z \sim N_p(0, I_p).$$

Then

$$R^2 = \|Z\|_2^2 = \sum_{j=1}^p Z_j^2 \sim \chi_p^2.$$

Thus

$$R \sim \chi_p.$$

Moreover,

$$U = \frac{Z}{R}$$

is uniform on the sphere and independent of R .

Hence

$$\boxed{Z = RU}.$$

Simulating uniform directions in R

```
runif_sphere <- function(n, p) {  
  Z <- matrix(rnorm(n * p), nrow = n, ncol = p)  
  radius <- sqrt(rowSums(Z^2))  
  Z / radius  
}  
U <- runif_sphere(5000, 3)  
summary(rowSums(U^2))
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	1	1	1	1	1	1

Coordinate moments on the sphere

By symmetry,

$$E(U_j) = 0.$$

Also

$$\sum_{j=1}^p U_j^2 = 1.$$

Taking expectations,

$$\sum_{j=1}^p E(U_j^2) = 1.$$

All coordinates have the same second moment, so

$$\boxed{E(U_j^2) = \frac{1}{p}}.$$

For $j \neq k$, symmetry gives

$$E(U_j U_k) = 0.$$

Thus

$$E(UU^\top) = \frac{1}{p}I_p.$$

R TASK: check the sphere identities

Generate 100 000 observations uniformly on $\mathbb{S}^4$, i.e. use $p = 5$.

Check numerically:

$$E(U_j) \approx 0,$$

$$E(U_j^2) \approx \frac{1}{5},$$

and

$$E(UU^\top) \approx \frac{1}{5}I_5.$$

Solution: sphere identities

```
set.seed(20260812)
U5 <- runif_sphere(100000, 5)
colMeans(U5)
```

```
## [1] -0.0003973690  0.0006644818  0.0027927862  0.0003794449 -0.0006083274
```

```
colMeans(U5^2)
```

```
## [1] 0.2002711 0.1987351 0.2006800 0.1993509 0.2009630
```

```
crossprod(U5) / nrow(U5)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.2002710808 -0.0005010351 -1.884360e-04 -1.469934e-04 -6.633971e-04
## [2,] -0.0005010351  0.1987350649  1.119059e-03 -3.560403e-04 -4.687094e-04
## [3,] -0.0001884360  0.0011190594  2.006800e-01  4.011934e-05 -1.901243e-04
## [4,] -0.0001469934 -0.0003560403  4.011934e-05  1.993509e-01 -8.962631e-05
## [5,] -0.0006633971 -0.0004687094 -1.901243e-04 -8.962631e-05  2.009630e-01
```

The diagonal entries should be close to $1/5$, and the off-diagonal entries close to zero.

Elliptical distributions

From spheres to ellipsoids

Start from a spherical vector

$$Y = RU.$$

Apply a linear transformation A and location shift μ :

$$X = \mu + RAU.$$

This gives an **elliptical distribution**.

The components play different roles:

- ▶ U : direction;
- ▶ R : radial behaviour and tails;
- ▶ A : shape and orientation;
- ▶ μ : location.

Shape matrix

Let

$$\Sigma = AA^{\top}.$$

Then Σ describes the elliptical shape.

If A is nonsingular, surfaces with constant radial value correspond to

$$(x - \mu)^{\top} \Sigma^{-1} (x - \mu) = \text{constant}.$$

Thus spheres are transformed into ellipsoids.

Connection to Lecture 2

Lecture 2 gave several ways to factor a positive-definite matrix:

$$\Sigma = LL^\top$$

by Cholesky, or

$$\Sigma = Q\Lambda Q^\top.$$

A symmetric square root is

$$\Sigma^{1/2} = Q\Lambda^{1/2}Q^\top.$$

Both can be used to transform spherical random vectors.

Different square-root factors need not produce the same point for a fixed random input, but they can produce the same target distribution.

Multivariate normal from a spherical normal

Let

$$Z \sim N_p(0, I_p)$$

and choose A satisfying

$$AA^\top = \Sigma.$$

Then

$$X = \mu + AZ$$

satisfies

$$X \sim N_p(\mu, \Sigma).$$

Indeed,

$$E(X) = \mu$$

and

$$\text{Cov}(X) = AIA^\top = \Sigma.$$

For the normal family, the linear transformation also gives the full multivariate normal distribution.

Cholesky simulation

```
rmvnorm_chol <- function(n, mu, Sigma) {  
  p <- length(mu)  
  R <- chol(Sigma)  # Sigma = t(R) %*% R  
  Z <- matrix(rnorm(n * p), nrow = n, ncol = p)  
  X <- Z %*% R  
  sweep(X, 2, -mu, FUN = "-")  
}  
Sigma <- matrix(c(2, 1.2, 1.2, 1), 2, 2)  
X <- rmvnorm_chol(10000, c(1, -1), Sigma)  
  
colMeans(X)
```

```
## [1] 1.0043669 -0.9971579
```

```
cov(X)
```

```
##           [,1]      [,2]  
## [1,] 1.984629 1.198541  
## [2,] 1.198541 1.003619
```

Normal simulation via radius and direction

We can also build the standard normal explicitly from spherical components:

1. generate

$$U \sim \text{Unif}(\mathbb{S}^{p-1});$$

2. generate

$$V \sim \chi_p^2;$$

3. set

$$Z = \sqrt{V} U.$$

Then

$$Z \sim N_p(0, I_p).$$

Finally,

$$X = \mu + AZ$$

gives the desired multivariate normal.

Why the radial distribution matters

Two elliptical distributions can have the same

$$\mu \quad \text{and} \quad AA^\top$$

but very different tails because their radial variables R differ.

Thus covariance or shape alone does not determine an elliptical distribution.

The normal distribution corresponds to the special radial law

$$R^2 \sim \chi_p^2.$$

Heavy-tailed radial laws produce heavy-tailed elliptical distributions.

A general elliptical simulator

```
relliptical <- function(n, mu, A, rfun) {  
  p <- length(mu)  
  U <- runif_sphere(n, p)  
  radius <- rfun(n)  
  Y <- (U * radius) %*% t(A)  
  sweep(Y, 2, -mu, FUN = "-")  
}
```

The simulation algorithm separates **direction**, **radius**, and **linear transformation**.

This makes the geometry of the model explicit.

Summary

- ▶ Monte Carlo replaces difficult expectations or probabilities by empirical averages.
- ▶ LLN gives consistency; the CLT gives Monte Carlo uncertainty.
- ▶ Monte Carlo error decreases only as $m^{-1/2}$.
- ▶ Simulation studies can estimate bias, MSE, coverage and power.
- ▶ Importance sampling changes the simulation distribution to improve efficiency.
- ▶ Spherical distributions separate radius and direction.
- ▶ A normalized standard normal vector is uniform on the sphere.
- ▶ $N_p(0, I)$ has radius squared χ_p^2 .
- ▶ Elliptical distributions arise from $X = \mu + RAU$.
- ▶ Multivariate normal simulation connects directly to Cholesky and eigenvalue decompositions from Lecture 2.